

One Substrate: Byte-Exact Local Maintenance of a Curvature-Weighted Graph Laplacian for Interactive Rendering and Physics

Anonymous submission (double-blind review)

Abstract—Interactive engines keep rendering and physics in separate data structures; geometry processing reuses prefactored Laplacians but assumes fixed topology. We present a system in which a single Ollivier–Ricci-curvature-weighted graph Laplacian Δ is the shared substrate for rendering signals and four physics solvers (wave, heat, frequency-domain Helmholtz acoustics, and vector diffusion), maintained *incrementally and byte-identically* under structural double-pushout (DPO) graph rewrites. We observe that the standard globally-formulated Ollivier–Ricci computation is not byte-reproducible under edits—the W_1 transport LP’s floating-point output depends on global support size—and show that a bounded-support curvature κ_R ($R=2$) restores *exact* locality: after an edit, recomputing only a 2-hop dirty cover derived from the DPO interface yields an operator bit-identical to a full rebuild, by construction. Indeed, in exact arithmetic κ_R equals the global curvature—the bounded support is not an approximation but the same optimum computed over an edit-invariant certificate. A Mayer–Vietoris residual over the graph’s flag complex certifies each localized rebuild at runtime. On lattice and irregular triangle-mesh benchmarks the localized rebuild recomputes a constant number of edges as meshes grow ($\text{dirty}/|E| \rightarrow 0.002$), bit-exactly on every measured edit; with a sparse assembly of the same bit-identical entries the per-edit rebuild stays $\sim 150\times$ below a full rebuild at every measured size. Certificate overhead starts at ~ 1 ms, and the whole engine—editor with undo/redo included—is deterministic to the bit.

I. INTRODUCTION

Real-time engines treat the render representation and the physics representation as separate worlds synchronized by ad-hoc glue; recent work on physics for 3D Gaussian splats still opens by noting that “production-grade physics engines do not understand” the render representation [1]. Geometry processing, by contrast, has long organized many tasks around one prefactored operator — discrete exterior calculus [2], [3], the heat method [4], the vector heat method [5], Laplacian mesh editing [6], and projective dynamics [7] all “factor once, reuse many” — but on a *fixed* mesh: structural change means refactoring from scratch.

We do not introduce a new curvature, a new Laplacian, or a new incremental factorization; we show that an Ollivier–Ricci-weighted graph Laplacian can serve as a single, *dynamically-maintained* substrate for both rendering and several physics solvers, and that under DPO structural edits this substrate can be rebuilt locally, byte-identically,

and self-certified at runtime by a Mayer–Vietoris residual — turning determinism into a correctness proof for a live, editable engine.

Concretely, we contribute:

- 1) **An observation** (motivation, not a theorem about ORC): under the global W_1 LP formulation (Charnes perturbation over the full component), the *computed* curvature of far-away edges drifts ($\sim 10^{-10}$) when a node is added anywhere in the component. The mathematical locality of ORC [8] does not transfer to the computed value (Observation 1 gives the exact injected-mass mechanism); a byte-exact engine cannot use the global formulation.
- 2) **Bounded-support κ_R with exact locality**: $\kappa_R(e)$ is a pure function of the induced radius- R ball $B_R(e)$; with $R=2$ the in-ball metric restricted to the transport support coincides with the global metric (Lemma 1), in exact arithmetic κ_R equals the global curvature (Lemma 2), and the localized rebuild after a DPO rewrite is bit-identical to a full rebuild (Theorem 1).
- 3) **A runtime cohomological certificate**: a Mayer–Vietoris residual over the rewrite’s $U/K/R$ cover of the flag complex, evaluated per edit in a live engine. We claim no primacy on incremental cohomology, bit-exact maintenance, or MV-as-certificate — all have close prior art [9], [10]; §II delimits the claim to its defensible core: the first cohomological certificate acting as an *external witness* guarding a *curvature-weighted rendering/physics operator’s* localized rebuild under *DPO scene-graph edits* in a *live, deterministic, interactive engine*.
- 4) **The one-substrate system**: the same cached kernel-aware $LDL^\top$ of Δ drives light/importance, wave, heat, Helmholtz acoustics (indefinite, via Bunch–Kaufman on the k^2 -shifted copy), vector diffusion, and unconditionally stable implicit stepping, in a deterministic interactive editor with undo/redo, gated by TLA^+ -mirrored invariants and byte-exact CI oracles.

II. RELATED WORK

Ollivier–Ricci curvature and its locality.: ORC compares 1-hop transport measures [11]; its locality is established — Jost and Liu bound it by degrees and short-cycle counts [8], and exact computation reduces to a local matching problem with $\Omega(n)$ worst-case query bounds for global formulations [12]. Ricci flow pipelines recompute

ORC only near modified edges [13]. We claim no locality discovery: κ_R is an engineering realization of known locality, chosen so the *computed* value is a pure function of an edit-invariant ball.

Incremental Laplacian factor reuse.: Local reuse of prefactored sparse systems under mesh edits is an established line: sub-mesh factor reuse [14], localized solves [15], and interactive sparse Cholesky updates [16]; dynamic Laplacian solvers exist in theory [17]. These optimize speed with bounded residual; none guarantees — or aims at — bit-identity with a from-scratch rebuild, and none derives its update region from a rewrite calculus. Our delta is the guarantee (Theorem 1) and the DPO-derived dirty cover (Lemma 3).

Sheaf cohomology, incrementally and as a certificate.: Spectral sheaf theory is due to Hansen and Ghrist [18]. Volk [9] is the closest algorithmic precedent for incremental cohomology maintenance, and we do not claim priority over it. His method maintains first sheaf cohomology on a streaming one-dimensional complex in time independent of complex size under bounded local geometry, and — beyond an amortized-cost result — proves the maintained state “agrees exactly with batch recomputation,” reporting “zero measured drift” through five million vertices; it even assembles the sheaf Laplacian $(\delta^0)^\top \delta^0$ and localizes injected contradictions by recomputing a single cell. Our contribution is orthogonal in what the cohomology is *for*: Volk re-verifies a maintained cohomological quantity against its own batch value in an offline stream; our Mayer–Vietoris residual is an external witness certifying the byte-exact rebuild of a curvature-weighted rendering-and-physics Laplacian under DPO scene-graph edits inside a live, deterministic editor — a different object, a different guarantee, a live setting.

Young [10] recasts program analysis as Čech cohomology of a semantic presheaf, where H^0 is “the space of globally consistent typings” and H^1 “classifies gluing obstructions,” with a Mayer–Vietoris sequence supplying “compositional obstruction counting, enabling incremental analysis.” Because that work already treats cohomology as an obstruction certificate and already uses Mayer–Vietoris for incrementality, we claim neither idea as novel in the abstract; what is new here is the target and the setting — an MV residual over a visibility sheaf on a flag complex that guards a numerical operator’s localized rebuild at interactive rates.

DPO rewriting in geometry; curvature-weighted Laplacians; determinism.: Algebraic graph transformation is classical [19]; topology-based geometric modelers rewrite generalized maps [20], but no prior work couples DPO rewrites to an incremental simulation operator. Curvature-weighted graph Laplacians appear in spectral clustering [21] and Ricci-flow edge weights [13] — network analysis, not rendering or physics. Bit-exact determinism is standard lockstep engineering [22], [23], [24]; our use of it as a per-edit correctness oracle (incremental $\equiv$ full, checked bitwise) is, to our knowledge, new in this setting.

III. SYSTEM OVERVIEW

The scene is a typed graph $G = (V, E, \tau, \alpha)$ with ten well-formedness invariants, specified in TLA^+ and mirrored token-exactly in the implementation (a regression test parses both). Edits are DPO rewrites — four verified rules (`spawn_light`, `remove_object`, `replace_material`, `refine_cell`) plus editor operations lowered to rewrites (add/delete object with cascade semantics, transactional OBJ import). All render and simulation products are pure functions of the graph: a lowering pass produces the raster and path-tracing scenes; the flow layer builds $\Delta = D_w - A_w$ over the Adjacent 1-skeleton with $w(e) = 1 + \max(\kappa(e), -0.95)$; the simulation layer evolves sections (scalar and vector fields) on Δ ; undo/redo replays the graph.

IV. EXACT LOCALITY OF BOUNDED-SUPPORT CURVATURE

A. Setup

For a scene graph G , the *skeleton* $\Sigma(G) = (V, E)$ consists of the mesh vertices (sorted, deduplicated) and the canonical Adjacent edge list (sorted vertex pairs, sorted lexicographically). d_G is the hop metric on $\Sigma(G)$, $N(v)$ the skeleton neighbourhood, and for an edge $e = (a, b)$, $B_R(e) = \{v : \min(d_G(v, a), d_G(v, b)) \leq R\}$, with $G[B_R(e)]$ the labeled induced subgraph. μ_v is uniform on $N(v)$; $\kappa(e) = 1 - W_1(\mu_a, \mu_b)/d_G(a, b)$. The *computed* κ_R executes: radius- R BFS ball, canonical sort, induced adjacency in canonical edge order, in-ball all-pairs BFS metric, support restriction, and a transportation simplex with the Charnes right-hand-side perturbation $\mu_i \mapsto \varepsilon(i+1)$, $\varepsilon = 10^{-12}$, over the support of size n_s — in **f64**, single-threaded, no randomness.

Determinism assumptions (standing). One binary, one ISA, no runtime FP-environment changes, single-threaded evaluation. Under IEEE-754 every operation is a deterministic function of operand bits, so a fixed instruction sequence on identical inputs yields identical output bits (the deterministic-lockstep setting [22], enforced in CI by byte-golden oracles).

B. Metric and value fidelity

Lemma 1 (metric fidelity of the 2-ball). *Let $e = (a, b) \in E$, $R \geq 2$, and $x, y \in N(a) \cup N(b)$. Then $d_{G[B_R(e)]}(x, y) = d_G(x, y)$.*

Proof. Write $B = B_R(e)$, $H = G[B]$. Since H is a subgraph, $d_H \geq d_G$. For the converse, note $d_G(x, y) \leq 3$ via the walk $x a b y$, so $d_G(x, y) \in \{0, 1, 2, 3\}$, and $x, y \in B_1 \subseteq B$. Take any G -geodesic P from x to y . If $|P| \leq 1$ its elements are in B . If $P = x m y$, then m is adjacent to $x \in B_1$, so $m \in B_2 \subseteq B$. If $P = x m_1 m_2 y$, then m_1 is adjacent to x and m_2 to y , so $m_1, m_2 \in B_2$. In every case all vertices of P lie in B , its edges are induced, and P is an H -path of length $d_G(x, y)$. $\square$

Lemma 2 (exact-value fidelity). *In exact arithmetic with the unperturbed transport LP, $\kappa_R(e) = \kappa(e)$ for every*

$e \in E$ and every $R \geq 2$: the bounded support is not an approximation.

Proof. Both formulations minimize $\sum_{ij} \pi_{ij} c_{ij}$ over plans with marginals μ_a, μ_b , differing only in the variable index set (global component support vs. ball support) and the metric (d_G vs. d_H). Mass conservation forces $\pi_{ij} = 0$ off $\text{supp } \mu_a \times \text{supp } \mu_b$, so both problems equal the problem posed on the supports alone. For $R \geq 1$ the in-ball star of a equals its true star ($N(a) \subseteq B_1$, and each star edge is induced), so both formulations carry the same marginals; by Lemma 1 the costs among support points coincide; and $d_G(a, b) = d_H(a, b) = 1$. $\square$

Corollary 1. *Observed discrepancies $|\kappa_R - \kappa|$ ($\leq 10^{-6}$ in our test suite, $\sim 10^{-10}$ typical) are entirely perturbation and floating-point artifacts; model error is exactly zero.*

Observation 1 (support-size dependence of the computed value). The implementation perturbs the LP right-hand side by $\mu_i += \varepsilon(i+1)$, $i = 1..n_s$, and $\nu_j += \varepsilon(n_s+1)/2$, injecting total extra mass $\varepsilon n_s(n_s+1)/2$ that the optimal plan must transport. The returned value therefore depends on the support size n_s and on each support point’s index. Under the global formulation n_s is the component size: inserting *any* vertex or edge moves the computed value of every edge in the component at order $\varepsilon n_s \cdot \text{diam}$ (measured $\sim 10^{-10}$). This does not contradict mathematical locality (Lemma 2); it obstructs byte-exact caching. Under the ball formulation, n_s and the indices are functions of the ball alone.

C. Byte-exact locality and the dirty cover

Proposition 1 (byte-exact locality). *Fix $R \geq 1$ and the standing determinism assumptions. If $B_R(e)$ computed in G and in G' coincide as node-id sets and the labeled induced subgraphs are equal, then the computed $\kappa_R(e; G)$ and $\kappa_R(e; G')$ are bit-identical.*

Proof sketch. Every intermediate artifact is a pure function of the labeled ball: the BFS ball set is order-invariant and self-contained (every witnessing path lies in the ball); the ball order is its sorted id list; the induced adjacency lists inherit the canonical edge order, whose in-ball subsequence is intrinsic to the ball; the in-ball metric, support, measures, and the fixed-pivot perturbed simplex consume only those artifacts. Identical inputs to a fixed **f64** instruction sequence produce identical bits. The full step-by-step argument is in the supplemental proofs.¹ $\square$

Lemma 3 (dirty-cover soundness). *Let a localizable edit take $G \rightarrow G'$, let the seed set S contain every created skeleton vertex and every endpoint of a created or deleted skeleton edge that exists in $\Sigma(G')$, and let $D = \{(u, v) \in E(G') : u \text{ or } v \text{ within distance } R \text{ of } S \text{ in } \Sigma(G')\}$. Then for every $e = (a, b) \in E(G') \setminus D$, the labeled R -balls of e in G and G' coincide; hence by Proposition 1 the cached $\kappa_R(e; G)$ is bit-identical to $\kappa_R(e; G')$.*

¹docs/paper/proofs.md in the artifact.

Proof. Suppose $e \notin D$. (i) *No vertex enters the ball*: if $v \in B_R(e)^{G'} \setminus B_R(e)^G$, a G' -path P of length $\leq R$ from $\{a, b\}$ to v must contain a created edge; the nearer endpoint u of the *first* created edge along P is a seed, and the prefix of P shows $d_{G'}(u, \{a, b\}) \leq R$, so $e \in D$ — contradiction. (ii) *No vertex leaves*: if $v \in B_R(e)^G \setminus B_R(e)^{G'}$, a G -path of length $\leq R$ must contain a deleted edge; the prefix up to the first deleted edge contains no deleted edge, hence survives into $\Sigma(G')$ (surviving edges keep surviving endpoints), so its endpoint u is a live seed with $d_{G'}(u, \{a, b\}) \leq R$ — contradiction. (iii) *No induced edge changes*: a created or deleted edge (u, w) with u, w in the common ball puts the live seed u within R of $\{a, b\}$ — contradiction. Created isolated vertices lie in no ball; deleted vertices are covered by (ii) via their cascaded edge deletions. $\square$

Theorem 1 (bitwise equality of localized and full rebuild). *Let $G_0 \rightarrow G_1 \rightarrow \dots \rightarrow G_k$ be a trace of localizable edits, with Δ_0 the full bounded build on G_0 and Δ_t the localized rebuild on G_t from Δ_{t-1} with the dirty set of Lemma 3. Then every Δ_t is bit-identical (matrix and curvature map) to the full bounded build on G_t .*

Proof. Induction on t ; $t=0$ is definitional. For the step, each skeleton edge of G_t is either dirty or a cache miss — recomputed by the identical primitive on the identical shared adjacency the full build uses, hence bit-equal — or a clean cache hit, whose stored value equals the full build’s value on G_{t-1} (inductive hypothesis) and hence, by Lemma 3 and Proposition 1, the full build’s value on G_t . Both paths then insert the identical map in the identical canonical order and run the identical assembly summation, so the matrices agree bitwise. $\square$

Remark 1. Theorem 1 is enforced, not just proven: continuous integration asserts == on every matrix entry and curvature across multi-edit traces, and the benchmark harness asserts it on every measured row of both benchmark families (§VII).

V. THE RUNTIME MAYER–VIETORIS CERTIFICATE

Each localizable rewrite is decomposed into subgraphs (U, K, R) (survivors, interface, replaced region). The engine builds the flag complexes of G' , U , R , K , the visibility sheaf on each, and checks the signed residual

$$h^0(G') - (h^0(U) + h^0(R) - h^0(K) + \varepsilon_{\text{sheaf}}) = 0,$$

where $\varepsilon_{\text{sheaf}}$ is the rank of the connecting map. A zero residual certifies the cover that Lemma 3 decomposes over — an external witness for the operator rebuild, evaluated per edit on the live graph (cost: §VII).

VI. ONE SUBSTRATE, FIVE CONSUMERS

The renderer’s importance channel reads the curvature signal; wave, heat, and component-wise vector diffusion are explicit matvec steppers on Δ ; Helmholtz acoustics solves $(\Delta - k^2 I)\varphi = s$, reusing the cached kernel-aware $LDL^\top$ in the PSD regime and Bunch–Kaufman factoring the shifted copy in the indefinite regime; and implicit backward-Euler

steppers factor $I + \beta\Delta$ (SPD for all $\beta \geq 0$) — densely via the kernel-aware $LDL^\top$ or, from the sparse carrier, via a sparse $LDL^\top$ (Davis-style elimination tree, deterministic natural ordering), making the implicit pipeline sparse end-to-end: CSR assembly $\rightarrow O(\text{nnz})$ shift $\rightarrow$ sparse factor $\rightarrow$ sparse solve. Unconditional stability: our tests drive $100\times$ past the explicit CFL bound, where the explicit stepper provably diverges, and assert bounded, mass-conserving, bit-deterministic trajectories. All consumers share the one maintained operator; none owns a private copy of the scene.

VII. EVALUATION

All numbers are 5-repetition medians from the deterministic benchmark harness; every row asserts the bit-exactness of Theorem 1 (the harness exits nonzero on any mismatch — all sweeps ran green), and every figure regenerates from committed CSVs.

Lattice family.: $R \times R$ 4-neighbour lattices, $|V|$ 64–4096, with a per-grid edit trace (1-edge add, 2-edge add, corner add, cascade delete) threading the previous operator exactly as the editor does. Figure 1 (top): the full bounded rebuild grows linearly in $|E|$ ($\sim 15\text{ ms} \rightarrow 4.9\text{ s}$); the dense-assembly localized rebuild stays flat while the dirty cover is constant (37–51 edges for interior edits, 13 at the corner, *at every size*) until the $O(n^2)$ assembly dominates (knee at $|E| \approx 2\text{ k}$, $\sim 130\text{ ms}$ at 4k nodes, $37\times$). The *sparse*-assembly localized rebuild — same dirty cover, operator entries bit-identical — removes the knee: 22–42 ms across grids 32–64, i.e. $\sim 152\times$ over the full rebuild at 4k nodes, with only the linear $O(V+E)$ skeleton scan remaining. dirty/ $|E|$ falls from 0.33–0.44 to 0.0016–0.0063 and never approaches the engine’s 0.5 fallback gate beyond toy sizes.

The global-formulation contrast case.: The globally-supported build takes 6.1 s at 64 nodes and 337 s at 144 nodes, versus 20 ms and 84 ms bounded ($309\times$, $4005\times$) — it is unmeasurable past toy sizes, which is precisely why a byte-exact engine needs κ_R (Observation 1).

Irregular family.: Face-adjacency graphs of perforated icospheres (interior degree 3, hole boundaries 1–2, pentagonal clusters), 78–5023 faces, plus an *-obj* mode replaying the trace on arbitrary OBJ meshes. The same invariants hold: constant dirty cover (15–28 edges — tighter than the lattice’s, as triangle adjacency has smaller 2-hop balls), dirty/ $|E|$ down to 0.0020, bit-exact throughout. Dense-assembly local plateaus at $\sim 83\text{ ms}$ at 5k faces ($14\times$); sparse-assembly local stays at 7–8 ms ($\sim 160\times$ over the full rebuild).

Factor family.: Factoring $I + \beta\Delta$ for the implicit steppers, dense vs sparse, on the lattices: the dense $LDL^\top$ climbs as $O(n^3)$ — 0.4 ms, 2.9 s, 19.3 s, and 261 s at $|V| = 64, 1024, 2304, 4096$ — while the sparse factor takes 0.13, 7.8, 33.5, and 112.6 ms respectively ($2317\times$ at 4k nodes); solves are $465\times$ faster (0.24 ms vs 113 ms). Solutions from the two factor paths agree within 10^{-9} at every size (a benchmark gate, not an observation). Per-edit implicit-stepper refactoring is therefore interactive at every measured scale.

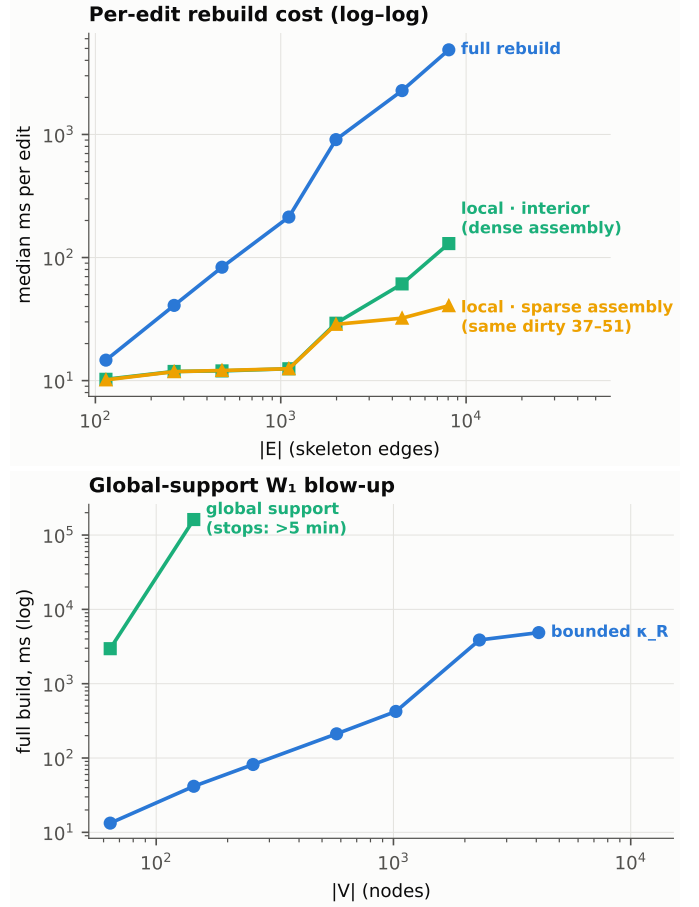


Fig. 1. Top: per-edit operator rebuild cost on lattices (log-log). The dense-assembly localized rebuild is flat until the $O(n^2)$ assembly dominates; the sparse assembly (same dirty cover, bit-identical entries) removes it. Bottom: the global-support W_1 formulation blows up (337 s at 144 nodes) while the bounded operator scales.

Certificate cost.: The MV residual is 0 on every edit of every sweep; cost $\sim 1\text{ ms}$ at 65 nodes, $\sim 7.7\text{ ms}$ at 257, $\sim 53\text{ ms}$ at 577 (superlinear; the visibility-sheaf H^0 over the full flag complex dominates — localizing the certificate is future work, though it is already viable per-edit at editor scale).

Verification stack.: 626 test cases / 84k assertions across unit, Tier-1 byte-exactness ($==$ on every entry over multi-edit traces), Tier-2 MV, and golden PPM regression baselines; AddressSanitizer+UBSan gate; TLC model checking of the graph and DPO-rule specifications with a $\text{TLA} \leftrightarrow \text{C++}$ token synchronization test.

VIII. LIMITATIONS AND FUTURE WORK

The sparse CSR assembly ($O(V + E \log \deg)$, entries bit-identical to the dense path) removes the assembly bound visible as the dense knee in Fig. 1, and the implicit path’s $I + \beta\Delta$ factors now go through a sparse $LDL^\top$ (§VII). What remains dense: the Helmholtz *indefinite* Bunch–Kaufman (sparse symmetric-indefinite factorization, where pivoting fights the symbolic structure, is a research problem of its own) and the cached component of the incremental solver that Helmholtz reads; fill-

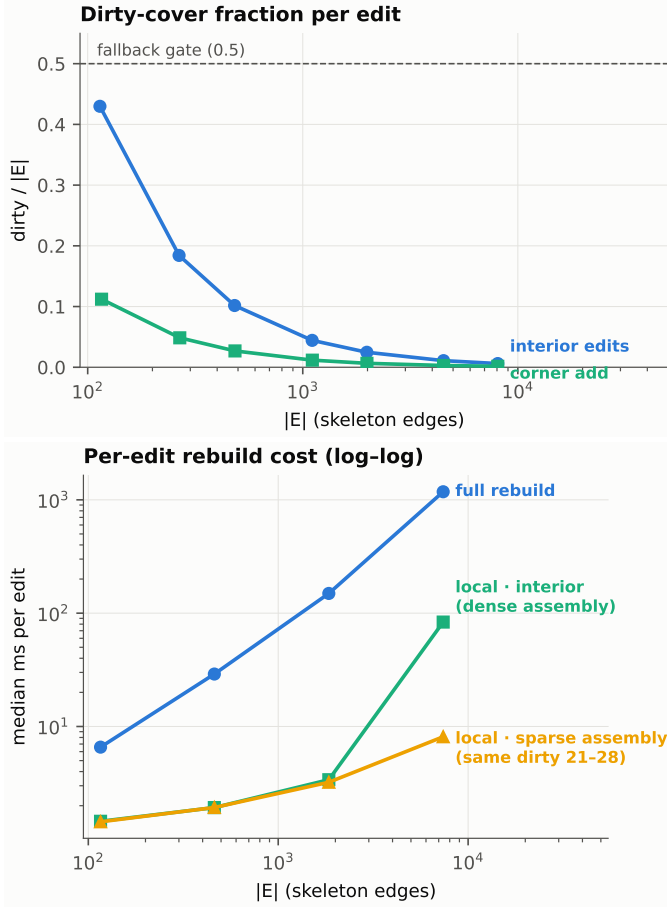


Fig. 2. Top: dirty-cover fraction per edit vanishes as meshes grow; the 0.5 fallback gate is never approached beyond toy sizes. Bottom: the irregular family (perforated-icosphere face-adjacency graphs) reproduces the lattice behaviour with a tighter constant cover (15–28 edges).

reducing orderings are future work. One honest floating-point caveat, encoded in our tests: matvecs *across* carriers agree to a few ulps and factor-path solves to 10^{-9} — not bitwise (contraction and elimination-order discretion); each path is individually byte-deterministic, and the operator entries are bit-identical. The rendering-importance channel still uses the global-support curvature (deliberately, as the contrast case). Implicit steppers refactor $I + \beta \Delta$ per (Δ, dt) ; factor *updates* under edits are future work. Component-wise vector diffusion does not couple components; a connection/Hodge Laplacian is the follow-up numerics. The certificate is global in G' ; localizing it would remove its superlinear cost.

IX. CONCLUSION

A curvature-weighted graph Laplacian can be the one substrate an interactive engine derives everything from — light, waves, heat, sound, vector fields — provided it can be maintained under structural edits without drift. We showed the standard global ORC formulation cannot support that (its computed values are support-size-dependent), that a bounded-support curvature makes locality *exact* rather than approximate, that the resulting

localized rebuild is bit-identical to a full rebuild with a machine-checked proof structure (per-edit CI equality) and a runtime cohomological witness, and that the whole stack holds up across two benchmark families at interactive rates. Determinism here is not netcode hygiene; it is the proof technique.

Reproducibility.: Single deterministic C++26 codebase; every number and figure regenerates from committed harnesses and CSVs; CI builds in a pinned gcc-16 container. **[TODO: artifact/code link after double-blind review]**

REFERENCES

- [1] X. Liu, S. Wu, and K. Han, “Scene-level heterogeneous physics simulation with 3d gaussian splats,” 2026, arXiv:2606.21753.
- [2] M. Desbrun, A. N. Hirani, M. Leok, and J. E. Marsden, “Discrete exterior calculus,” 2005, arXiv:math/0508341.
- [3] F. de Goes, K. Crane, M. Desbrun, and P. Schröder, “Digital geometry processing with discrete exterior calculus,” ACM SIGGRAPH 2013 Courses, 2013.
- [4] K. Crane, C. Weischedel, and M. Wardetzky, “Geodesics in heat: A new approach to computing distance based on heat flow,” *ACM Transactions on Graphics*, vol. 32, no. 5, pp. 152:1–152:11, 2013.
- [5] N. Sharp, Y. Soliman, and K. Crane, “The vector heat method,” *ACM Transactions on Graphics*, vol. 38, no. 3, pp. 24:1–24:19, 2019.
- [6] O. Sorkine, D. Cohen-Or, Y. Lipman, M. Alexa, C. Rössl, and H.-P. Seidel, “Laplacian surface editing,” in *Proceedings of the 2004 Eurographics/ACM SIGGRAPH Symposium on Geometry Processing (SGP)*. ACM, 2004, pp. 175–184.
- [7] S. Bouaziz, S. Martin, T. Liu, L. Kavan, and M. Pauly, “Projective dynamics: Fusing constraint projections for fast simulation,” *ACM Transactions on Graphics*, vol. 33, no. 4, pp. 154:1–154:11, 2014.
- [8] J. Jost and S. Liu, “Ollivier’s Ricci curvature, local clustering and curvature-dimension inequalities on graphs,” *Discrete & Computational Geometry*, vol. 51, no. 2, pp. 300–322, 2014.
- [9] J. L. Volk, “Incremental sheaf cohomology on cellular complexes: $O(1)$ -in- n lazy edit processing under bounded local geometry,” 2026, arXiv:2606.04227.
- [10] H. Young, “Sheaf-cohomological program analysis: Unifying bug finding, equivalence, and verification via čech cohomology,” 2026, arXiv:2603.27015.
- [11] Y. Ollivier, “Ricci curvature of Markov chains on metric spaces,” *Journal of Functional Analysis*, vol. 256, no. 3, pp. 810–864, 2009.
- [12] B. DasGupta, E. Grigorescu, and T. Mukherjee, “On computing discretized Ricci curvatures of graphs: local algorithms and (localized) fine-grained reductions,” *Theoretical Computer Science*, vol. 975, p. 114127, 2023.
- [13] C.-C. Ni, Y.-Y. Lin, F. Luo, and J. Gao, “Community detection on networks with Ricci flow,” *Scientific Reports*, vol. 9, 2019.
- [14] P. Herholz and M. Alexa, “Factor once: Reusing Cholesky factorizations on sub-meshes,” *ACM Transactions on Graphics*, vol. 37, no. 6, pp. 230:1–230:9, 2018.
- [15] P. Herholz, T. A. Davis, and M. Alexa, “Localized solutions of sparse linear systems for geometry processing,” *ACM Transactions on Graphics*, vol. 36, no. 6, pp. 183:1–183:8, 2017.
- [16] P. Herholz and O. Sorkine-Hornung, “Sparse Cholesky updates for interactive mesh parameterization,” *ACM Transactions on Graphics*, vol. 39, no. 6, pp. 202:1–202:14, 2020.
- [17] D. Durfee, Y. Gao, G. Goranci, and R. Peng, “Fully dynamic spectral vertex sparsifiers and applications,” in *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing (STOC 2019)*. ACM, 2019, pp. 914–925.
- [18] J. Hansen and R. Ghrist, “Toward a spectral theory of cellular sheaves,” *Journal of Applied and Computational Topology*, vol. 3, pp. 315–358, 2019.
- [19] H. Ehrig, K. Ehrig, U. Prange, and G. Taentzer, *Fundamentals of Algebraic Graph Transformation*, ser. Monographs in Theoretical Computer Science (EATCS Series). Springer, 2006.

- [20]H. Belhaouari, A. Arnould, P. Le Gall, and T. Bellet, “Jerboa: A graph transformation library for topology-based geometric modeling,” in *Graph Transformation — 7th International Conference, ICGT 2014, Held as Part of STAF 2014, York, UK, July 22–24, 2014. Proceedings*, ser. Lecture Notes in Computer Science, H. Giese and B. König, Eds., vol. 8571. Springer, 2014, pp. 269–284.
- [21]V. Kotharkar, “LLY ricci reweighting in stochastic block models: Uniform curvature concentration and finite-horizon tracking,” 2026, arXiv:2603.11060.
- [22]G. Fiedler, “Floating point determinism,” Gaffer On Games, Feb. 2010, https://gafferongames.com/post/floating_point_determinism/.
- [23]J. Demmel and H. D. Nguyen, “Fast reproducible floating-point summation,” in *2013 IEEE 21st Symposium on Computer Arithmetic (ARITH)*. IEEE, 2013, pp. 163–172.
- [24]W. Ahrens, J. Demmel, and H. D. Nguyen, “Algorithms for efficient reproducible floating point summation,” *ACM Transactions on Mathematical Software*, vol. 46, no. 3, pp. 22:1–22:49, 2020.